```
//Program on abstract class
package com.rsc.abstract_demo;

abstract class Animal
{
    protected String name;

    public Animal(String name)
    {
        super();
        this.name = name;
    }

    public abstract void run();
}

class Lion extends Animal
{
    public Lion(String lionName)
    {
        //invoke constructor of super class
        super(lionName.length() == 0 ? "null" : lionName.toLowerCase());
        System.out.println("Lion name cannot be null OR empty");
        System.exit(0);
    }
    super(lionName);
}

@Override
public void run()
{
    System.out.println(name+" Animal is running");
}
}

public class AbstractDemo2
{
    public static void main(String[] args)
    {
        String lionName = "";
        Lion lion = new Lion(lionName);
        animal.run();
    }
}
```

The following program explains that all the abstract methods of super class must be overridden in the sub classes. The following program explains that an abstract class never objects will not be created.

```
package com.rsc.abstract_demo;

abstract class Alpha
{
    public abstract void show();
    public abstract void demo();
}
abstract class Beta extends Alpha
{
    @Override
    public void show()
    {
        System.out.println("show method is overridden in Beta class");
    }
}
class Gamma extends Beta
{
    @Override
    public void demo()
    {
        System.out.println("demo method is overridden in Gamma class");
    }
}

public class AbstractDemo3
{
    public static void main(String[] args)
    {
        Gamma gamma = new Gamma();
        gamma.show();
        gamma.demo();
    }
}
}
```

We can throw an abstract method as a business rule that means an abstract method which is common method, must be overridden by all the sub classes (Implementation is compulsory) as shown in the program

```
package com.rsc.abstract_ex;

abstract class Animal
{
    protected String name;

    public Animal(String name)
    {
        this.name = name;
    }
    public abstract void regularHealthCheckup();
}

class Lion extends Animal
{
    public Lion(String name)
    {
        super(name);
    }

    @Override
    public void regularHealthCheckup()
    {
        System.out.println(" is going for regular health checkup");
    }

}

class Elephant extends Animal
{
    public Elephant(String name)
    {
        super(name);
    }

    @Override
    public void regularHealthCheckup()
    {
        System.out.println(" is going for regular health checkup");
    }

}

class Dog extends Animal
{
    public Dog(String name)
    {
        super(name);
    }

    @Override
    public void regularHealthCheckup()
    {
        System.out.println(" is going for regular health checkup");
    }

}

public class AbstractDemo4
{
    public static void main(String[] args)
    {
        Lion lion1 = new Lion("Simba: Abc");
        Elephant[] elephants = {new Elephant("Dumbo"), new Elephant("Ella")};
        Dog[] dogs = {new Dog("Jimmy"), new Dog("Sherry")};
        System.out.println("-----------------------");
        lion1.regularHealthCheckup();
        for(Elephant elephant : elephants)
        {
            elephant.regularHealthCheckup();
        }
        for(Dog dog : dogs)
        {
            dog.regularHealthCheckup();
        }
        System.out.println("-----------------------");
    }

    public static void regularHealthCheckup(Animal ...animals)
    {
        for(Animal animal : animals)
        {
            animal.regularHealthCheckup();
        }
    }
}
```

Anonymous Inner class (Method Implementation)

```
class Outer //Outer class
{
    private class Inner //Inner class
    {
    }
}
```

* **If** we define a class **inside a method body without any name then it is called Anonymous inner class.**

* The main purpose of this anonymous inner class is **extend a class** OR to **implemement an interface that means to create sub type for providing instant implementation.**
  [Anonymous inner class will work for Tolerance OR Interface implementation]

* The anonymous inner class, class body declaration (without name) and object creation by using new keyword, both are done in the **same line** at the time of declaration of Anonymous inner class.

* An Anonymous inner class must be terminated by ; (semicolon)

```
//Program :
//Anonymous inner class
//Anti Vehicle .//VehCls.class

    public void run()
    {
        SS.println("Generic Vehicle is Running");
    }
}

public class AllTerrain .//AllTerrain.class
{
    public static void main(String[] args)
    {
        //Anonymous inner class
        //Anti Vehicle .//AllTerrain$1.class
        Vehicle v1 = new Vehicle()
        {
            @Override
            public void run()
            {
                SS.println("Car Vehicle is Running");
            }
        };
        v1.run();

        //Anonymous inner class
        //Anti Vehicle = new Vehicle()//AllTerrain$2.class
        Vehicle v2 = new Vehicle()
        {
            @Override
            public void run()
            {
                SS.println("Bike Vehicle is Running");
            }
        };
        v2.run();

    }

}
```

```
package com.rsc.oc;

abstract class Shape
{
    public abstract void draw();

}

public class AllDemo2
{
    public static void main(String[] args)
    {
        Shape square = new Shape();
        {
            @Override
            public void draw()
            {
                SS.println("Drawing square");
            }
        };
        Shape rect = new Shape()
        {
            @Override
            public void draw()
            {
                SS.println("Drawing Rectangle");
            }
        };
        square.draw();  rect.draw();
    }

}
```